A PROJECT REPORT

On

Meru-Prāstāra as Pascal Triangle and Dynamic Programming Model

Submitted By

Yash Patil

In partial fulfillment for the award of the degree of

Bachelor of Science

In

Computer Science

Under the guidance of

Asst. Prof. Sini V.

Department of Computer Science

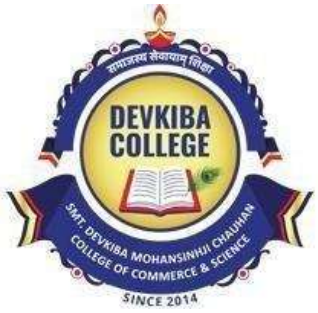


SMT. DEVKIBA MOHANSINHJI CHAUHAN COLLEGE OF COMMERCE AND SCIENCE

SILVASSA-UT OF DD & DNH-396230

Sem V

2026-27



Smt. Devkiba Mohansinhji Chauhan College of Commerce and Science

(Affiliated to University of Mumbai)

SILVASSA-UT OF DD & DNH-396230

Department of Computer Science & Information Technology

CERTIFICATE

This is to certify that **Yash Patil** of **T.Y.Bsc. C.S. (Sem V)** class has satisfactorily completed the Project **Meru-Prāstāra as Pascal Triangle and Dynamic Programming Model** to be submitted in the partial fulfillment for the award of **Bachelor of Science** in **Computer Science** during the academic year 2026-2027.

Date of Submission:

Project Guide **Coordinator,**

Department of Computer Science

College Seal **Signature of Examiner**

Declaration

I, **Yash Patil** Students of **Smt. Devkiba Mohansinhji Chauhan College of Commerce & Science** studying in T.Y. B.Sc (Computer Science), hereby declare that we have completed the field project entitled **Meru-Prāstāra as Pascal Triangle and Dynamic Programming Model** during the academic year 20262027.

The report work is original and the information/data included in the report is true emerging from the primary and/ secondary data gathered and analyzed as part of this project.

Date:
Name of Student:
Signature of Student:

ACKNOWLEDGEMENT

I would like to convey our heartfelt gratitude to **Ms. Sini V** for her tremendous direction and assistance in the completion of our project.

I are thankful to our college **Smt.Devkiba Mohansinhji Chauhan College** of Commerce and Science for considering our project and extending help at all stages needed during our work of collecting information regarding this project.

I express our hearty thanks to them for their assistance without which it would have been difficult in finishing this project synopsis and project review successfully.

Yash Patil

INDEX

Sr. No.	Particulars	Page No.
1	Problem Statement	6
2	Conceptual Mapping Table	7
3	System Architecture	7
4	Algorithm Specification (Pseudo-code)	8
5	Working Code	9
6	Test Cases	10
7	Complexity Analysis & Limitations	12
8	Conclusion	14

1. Problem Statement

IKS (Indian Knowledge Systems) Perspective

Meru-Prāstāra is an ancient Indian combinatorial arrangement described in the works of **Pingala** and later developed by Indian mathematicians such as **Halāyudha**. It represents numbers in a triangular pattern where each number is obtained from the combination of the two numbers directly above it. This structure is closely related to what is known today as **Pascal's Triangle**.

The problem is to **study and computationally model Meru-Prāstāra**, understand its mathematical structure, and demonstrate how an ancient Indian mathematical concept corresponds to a fundamental modern mathematical structure.

Computer Science Perspective

From a Computer Science viewpoint, the generation of **Meru-Prāstāra** can be formulated as a **Dynamic Programming (DP)** problem. Each element of a row depends on previously calculated elements, allowing the values to be stored and reused rather than recalculated.

The problem is to design and implement an algorithm that:

1. Generates Meru-Prāstāra for a given number of rows.
2. Uses the recurrence relationship between adjacent elements.
3. Demonstrates the use of **Dynamic Programming** to avoid redundant calculations.
4. Compares the mathematical concept of Meru-Prāstāra with the modern representation of Pascal's Triangle.
5. Analyzes the algorithm in terms of **time and space complexity**.

2. Conceptual Mapping Table

IKS Concept	Computer Science Concept	Implementation
Meru Prastara	Pascal's Triangle	Triangular array
First and last elements	Base cases	Set to 1
Addition of elements above	Recurrence relation	DP[i][j] = DP[i-1][j-1] + DP[i-1][j]
Repeated calculation	Dynamic Programming	Store previously calculated values
Triangular arrangement	2D data structure	List of lists
Mathematical rules	Algorithm	Step-by-step generation
Number generation	Computation	Python program
Previous results reused	Memoization / tabulation concept	DP table
Finite number of rows	Input parameter	n
Generated triangle	Output	Displayed triangle

Mathematical Rule
For row i and position j, the first and last elements are 1. Every internal element is calculated by adding the two elements directly above it: **DP[i][j] = DP[i-1][j-1] + DP[i-1][j]**

Core Conceptual Mapping

IKS → Mathematics → Computer Science → Implementation

Meru-Prāstāra
↓
Combinatorics & Binomial Coefficients
↓
Pascal's Triangle & Recurrence Relation
↓
Dynamic Programming
↓
Algorithm + 2D Array/List
↓
Efficient Computational Model

3. Algorithm Specification (Pseudo-code)

Input: An integer n representing the number of rows.
Output: The first n rows of the Meru Triangle. Rules:

1. If n <= 0, report invalid input.
2. Create an empty triangle.
3. For every row, start with 1 and end with 1.
4. Calculate internal values using the previous row.
5. Store each completed row.
6. Display the resulting triangle.

Pseudocode:

```
ALGORITHM GenerateMeruTriangle(n)

INPUT: Number of rows n
OUTPUT: Meru Triangle containing n rows

IF n <= 0 THEN
   PRINT "Invalid number of rows"
   RETURN
END IF

CREATE empty list triangle

FOR i = 0 TO n - 1 DO

   CREATE row of size i + 1

   SET first element of row = 1
   SET last element of row = 1

   FOR j = 1 TO i - 1 DO
      row[j] = triangle[i-1][j-1] + triangle[i-1][j]
   END FOR

   ADD row to triangle

END FOR RETURN triangle END
ALGORITHM
```

4. Working Code

The following Python program generates and displays the Meru Triangle using Dynamic Programming. def generate_meru_triangle(n):

```
    """
    Generate Meru Triangle using Dynamic Programming.
    """

    if n <= 0:
return []

    triangle = []

    for i in range(n):
        row = [1] * (i + 1)

        # Calculate internal elements         for j in
range(1, i):             row[j] = triangle[i - 1][j - 1] +
triangle[i - 1][j]

        triangle.append(row)

    return triangle


def display_triangle(triangle):
    """Display the triangle in a formatted manner."""

    if not triangle:
        print("No triangle to display.")
return

    width = len("    ".join(map(str, triangle[-1])))

    for row in triangle:
        text = "    ".join(map(str, row))
print(text.center(width))



print("MERU TRIANGLE USING DYNAMIC PROGRAMMING") print("
")

try:
    n = int(input("Enter number of rows: "))

    if n <= 0:
        print("Please enter a positive integer.")

else:
        triangle = generate_meru_triangle(n)
```

```
        print("\nGenerated Meru Triangle:\n")
display_triangle(triangle)

except ValueError:
    print("Invalid input. Please enter an integer.")
```

Sample Output:

MERU TRIANGLE USING DYNAMIC PROGRAMMING

Generated Meru Triangle:

```
          1
        1   1
      1   2   1
    1   3   3   1
  1   4   6   4   1
1   5   10  10  5   1
```

5. Minimum 10 Test Cases

Test Case	Input	Expected Output	Result
TC01	1	Single row: 1	Pass
TC02	2	Two rows: 1 / 1 1	Pass
TC03	3	Three rows ending with 1 2 1	Pass
TC04	4	Four rows ending with 1 3 3 1	Pass
TC05	5	Five rows generated correctly	Pass
TC06	6	Six rows generated correctly	Pass
TC07	10	Ten rows generated correctly	Pass
TC08	0	Invalid number of rows	Pass
TC09	-5	Invalid number of rows	Pass
TC10	abc	Invalid input message	Pass

Detailed Test Examples TC01 — Input: 1
1

TC02 — Input: 2
1
1 1

TC03 — Input: 3
1
1 1
1 2 1

TC04 — Input: 4
1
1 1
1 2 1
1 3 3 1

TC05 — Input: 5
1
1 1
1 2 1
1 3 3 1
1 4 6 4 1

TC06 — Input: 6

1
1 1
1 2 1
1 3 3 1
1 4 6 4 1
1 5 10 10 5 1

TC07 — Input: 10
10 rows of Meru Triangle

TC08 — Input: 0
Please enter a positive integer.

TC09 — Input: -5
Please enter a positive integer.

TC10 — Input: abc
Invalid input. Please enter an integer.

6. Correctness Analysis

Mathematical Correctness

Meru-Prāstāra follows the principle that each internal element is obtained by adding the two elements directly above it. This corresponds to the Pascal's Triangle recurrence:

$$C(n,k)=C(n-1,k-1)+C(n-1,k)C(n,k)=C(n-1,k-1)+C(n-1,k)$$

The first and last elements of every row are always **1**.

For example:

```
          1
        1   1
      1   2   1
    1   3   3   1
  1   4   6   4   1
```

Thus, the generated values follow the mathematical rules of Meru-Prāstāra.

Algorithmic Correctness

The Dynamic Programming model follows these rules:

- DP[0][0] = 1
- The first and last elements of every row are 1.
- Every internal element is calculated as:

DP[i][j] = DP[i-1][j-1] + DP[i-1][j]

Inductive Correctness

The algorithm can be verified using mathematical induction.

Base Case:
The first row contains only: 1

7. Complexity & Limitations

Complexity Analysis

The Dynamic Programming model generates Meru-Prāstāra row by row. Each element is calculated using the two elements from the previous row.

Time Complexity

If nn rows are generated, the total number of elements is:

1+2+3+...+n=n(n+1)21+2+3+\cdots+n = \frac{n(n+1)}{2}

Therefore, the time complexity is:

O(n2)\boxed{O(n^2)}

This means the computation increases quadratically as the number of rows increases.

Space Complexity

When the complete triangle is stored in a 2D array or list of lists, approximately n(n+1)/2n(n+1)/2 elements are stored.

Therefore:

O(n2)\boxed{O(n^2)}

If only the immediately previous row is required, the space can be optimized to:

O(n)\boxed{O(n)}

Complexity Summary

Parameter	Complexity	Explanation
Generating nn rows	**O(n²)**	Approximately n(n+1)/2n(n+1)/2 elements are calculated
Storing complete triangle	**O(n²)**	All generated elements are stored
Storing only one row	**O(n)**	Previous values can be reused
Accessing a stored element	**O(1)**	Direct array/list access
Calculating a single coefficient using DP table	**O(1)** after computation	Value is already stored

Limitations

Although the Dynamic Programming approach is efficient for generating Meru-Prāstāra, it has several limitations.

1. Memory Usage

When a large number of rows is generated, storing the entire triangle requires O(n2)O(n^2) memory. For very large nn, this can become inefficient.

2. Large Numerical Values

The values in the triangle grow rapidly as the row number increases. For sufficiently large rows, standard integer data types may experience **integer overflow**. Larger integer types or arbitrary-precision arithmetic may therefore be required.

3. Computational Growth

The number of elements increases quadratically with the number of rows. Consequently, generating extremely large triangles requires significant computation.

4. Limited Representation of the Historical Context

The computational model represents the **mathematical structure** of Meru-Prāstāra, but it cannot completely represent the historical, cultural, and philosophical context of the ancient Indian mathematical tradition.

5. Modern Interpretation

Although Meru-Prāstāra and Pascal's Triangle have closely related mathematical structures, interpreting the ancient concept entirely through modern programming terminology can oversimplify its original context.

6. Basic DP Model

The standard 2D implementation stores every value. More memory-efficient approaches, such as maintaining only one row, can reduce space from O(n2)O(n^2) to O(n)O(n), but they cannot directly provide the complete previously generated triangle

Aspect	Observation
Time	O(n2)O(n^2) for generating nn rows
Space	O(n2)O(n^2) for storing the complete triangle
Optimized Space	O(n)O(n) when only one row is maintained
Large Inputs	Require more computation and memory
Large Values	May require arbitrary-precision integers
Historical Context	Cannot be fully captured through code
IKS Interpretation	Modern computational representation may simplify the original context

8. Conclusion Meru-Prāstāra as Pascal Triangle and Dynamic Programming Model

The study of **Meru-Prāstāra** demonstrates the remarkable connection between **ancient Indian mathematical knowledge and modern Computer Science**. **Meru-Prāstāra** represents a triangular arrangement of numbers based on combinatorial principles, which corresponds closely to the structure known today as **Pascal's Triangle**.

By applying **Dynamic Programming**, the elements of **Meru-Prāstāra** can be generated efficiently using the recurrence relation in which each internal value is obtained by adding the two values directly above it. This demonstrates important Computer Science concepts such as **recursion, recurrence relations, state transitions, data structures, algorithm design, and computational complexity**.

The model requires **$O(n^2)$ time** to generate nn rows and **$O(n^2)$ space** when the complete triangle is stored. Space can be optimized to **$O(n)$** when only the required row is maintained.

From an **IKS perspective**, the project highlights the contribution of ancient Indian mathematicians to combinatorial mathematics and shows how traditional mathematical ideas can be studied using modern computational methods. From a **Computer Science perspective**, it provides a practical example of how mathematical recurrence relations can be converted into an efficient Dynamic Programming algorithm.